

# HashMap Internal Working - Interview Ready Notes

## 1. What is HashMap?

HashMap is a key-value data structure that uses hashing to store and retrieve data efficiently.

## 2. Internal Structure

HashMap internally uses an array called `table[]`. Each index is called a bucket.

Each bucket can store:

- null
- Linked List (Node)
- Red-Black Tree (TreeNode)

## 3. Node Structure

```
static class Node<K,V> {  
    final int hash;  
    final K key;  
    V value;  
    Node<K,V> next;  
}
```

## 4. Hash Calculation

`hash = key.hashCode() ^ (hash >>> 16)`

`index = (n - 1) & hash`

## 5. put() Flow

Step-by-step:

### 1. Compute hash

### 2. Find index

### 3. If empty → insert

### 4. If collision:

- Traverse linked list
- If key exists → update value
- Else → add new node

### 5. If nodes $\geq 8$ → convert to tree

### 6. Collision Handling

Collision occurs when multiple keys map to same index.

Handled using chaining (Linked List → Tree)

### 7. Linked List vs Tree

Linked List:  $O(n)$

Red-Black Tree:  $O(\log n)$

#### 8. Treeification Conditions

- Bucket size  $\geq 8$
- Table size  $\geq 64$

#### 9. Untreeify

If nodes  $< 6 \rightarrow$  convert back to Linked List

#### 10. Red-Black Tree Properties

- Root is black
- No two red nodes together
- Equal black height

#### 11. Resize & Rehash

Triggered when size  $>$  capacity \* loadFactor

Default load factor = 0.75

Capacity doubles, elements rehashed

#### 12. get() Flow

1. Compute hash
2. Find index
3. Traverse list or tree
4. Return value if found

#### 13. Time Complexity

Average:  $O(1)$

Worst (Linked List):  $O(n)$

Worst (Tree):  $O(\log n)$

#### 14. Interview Questions

- Why HashMap is not thread-safe?
- Difference between HashMap and ConcurrentHashMap?
- Why tree introduced in Java 8?
- What happens if all keys collide?

#### 15. Summary

HashMap uses array + linked list + red-black tree to handle collisions efficiently.